

Laboratorios de Sistemas Distribuidos

Guía del estudiante para las semanas 1 y 2

COMPUTACIÓN EN LA NUBE

Unidad 1 Introducción a los Sistemas Distribuidos

Octavo semestre de Ingeniería Informática

Prof. Jorge Rojas UCAB

Versión ampliada Septiembre de 2026

Propósito y alcance

Durante dos semanas construiremos y observaremos un sistema distribuido mínimo. La primera práctica muestra identidad, comunicación, balanceo, latencia y fallos parciales. La segunda introduce estado y permite comparar aislamiento, estado compartido, disponibilidad y límites de servicio.

El laboratorio no implementa consenso ni replicación. Su objetivo es producir evidencia que permita explicar por qué esos mecanismos existen y qué costos introducen.

Las etiquetas Temas de Unidad 1 usan la terminología de la presentación de la unidad. Replicación, consenso y CAP se analizan conceptualmente; este laboratorio no los implementa.

Resultados de aprendizaje

Resultado	Evidencia observable
Distinguir nodos y procesos	Campos node y pid en cada respuesta
Explicar la ruta de comunicación	Cliente a NGINX a web1 o web2 y luego a state
Reconocer latencia y fallo parcial	Tiempos, status HTTP y upstream en logs
Comparar estado local y compartido	Contadores divergentes frente a un valor común
Identificar un punto único de fallo	Respuesta 503 cuando state no está disponible
Analizar concurrencia y resultados inciertos	Ráfaga paralela, timeout de POST y reintento duplicado

Arquitectura del laboratorio



Compose crea una red privada y registra cada servicio en el DNS interno. NGINX resuelve web1 y web2 y distribuye las solicitudes. Las aplicaciones resuelven state cuando necesitan el contador compartido. Solo NGINX publica un puerto en el host.

Componente	Responsabilidad	Limitación deliberada
nginx	Entrada, request ID, balanceo y failover pasivo	No realiza health checks activos
web1 y web2	API HTTP y estado local por proceso	No comparten memoria
state	Serializa un contador común en memoria	No está replicado ni persiste datos

Entorno y preflight

Use Ubuntu 22.04 como camino principal. Ubuntu 20.04 puede utilizarse si Docker y Compose ya fueron instalados y validados antes de la clase. Node.js no se instala en el host: la imagen usa Node 24 LTS.

Requisitos

- Docker Engine en ejecución y permiso para usarlo.
- Docker Compose V2 recomendado o Compose V1 1.27 o posterior.
- curl disponible y puerto TCP 8080 libre.
- Acceso a node:24-bookworm-slim y nginx:1.28-alpine o imágenes precargadas.

Paso 1 Ubíquese en el proyecto

```
cd ucab-distributed-lab
```

Paso 2 Valide el host y la configuración

```
./scripts/preflight.sh
```

Resultado esperado El script termina con el mensaje de preflight completado.

Paso 3 Valide el archivo de Compose

```
docker compose config
```

Resultado esperado Compose muestra la configuración normalizada sin errores.

Paso 4 Construya e inicie el sistema

```
docker compose up --build -d
```

Resultado esperado Los cuatro servicios se crean y quedan iniciados.

Paso 5 Revise el estado de los servicios

```
docker compose ps
```

Resultado esperado nginx, web1, web2 y state aparecen en ejecución; los health checks pueden tardar unos segundos.

Si el equipo solo dispone de Compose V1, sustituya docker compose por docker-compose. No mezcle ambas variantes en una misma ejecución.

Estructura del repositorio

Archivo	Uso
app.js	API web, identidad y contadores
state-service.js	Servicio único de estado
docker-compose.yml	Topología y health checks
compose.delay.yml	Overlay de latencia para web1
nginx.conf	Balanceo, timeouts, DNS y logs
test/app.test.js	Tres pruebas automatizadas
scripts/preflight.sh	Validación previa del host
scripts/smoke-test.sh	Validación integral del despliegue

Código y contratos observables

El repositorio es la fuente de verdad del código. Antes de experimentar, identifique los contratos HTTP que utilizará para reunir evidencia.

Método y ruta	Respuesta esperada	Concepto
GET /	message, node, pid, timestamp y requestId	Identidad y trazabilidad
GET /health	status ok y node	Estado del proceso
GET o POST /counter/local	node y counter	Estado aislado
GET o POST /counter/shared	node y counter o error 503	Estado remoto compartido

Pruebas antes del laboratorio

Paso 1 Ejecute las pruebas unitarias

```
docker compose run --rm --no-deps web1 npm test
```

Resultado esperado El resumen informa tres pruebas aprobadas.

Paso 2 Ejecute la prueba integral

```
./scripts/smoke-test.sh
```

Resultado esperado El script confirma ambos nodos y el contador compartido.

El smoke test espera a que el servicio responda, verifica que ambos nodos reciban trabajo y comprueba una escritura y lectura del contador compartido.

Semana 1 Identidad comunicación y fallos

El objetivo de esta semana es observar dos procesos autónomos detrás de un punto de entrada común. Guarde las salidas relevantes y no asuma que una secuencia corta representa una regla.

Experimento A Identidad y procesos

Temas de Unidad 1 Definición de sistema distribuido | Autonomía | Procesos hilos y concurrencia

Paso 1 Envíe diez solicitudes

```
for i in $(seq 1 10); do
  curl -s http://localhost:8080/
  echo
done
```

Resultado esperado Las respuestas incluyen los nombres web1 y web2.

- ¿Cuántos nodos atienden solicitudes y qué campo los distingue?
- ¿Por qué dos contenedores pueden mostrar el mismo número PID?
- ¿Qué componente decide el destino de cada solicitud?

Experimento B Comunicación

Temas de Unidad 1 Comunicación distribuida | HTTP y serialización | Naming y discovery

Paso 1 Observe el proxy

```
docker compose logs --tail=30 nginx
```

Resultado esperado Cada línea identifica request ID, upstream, status y tiempos.

Paso 2 Observe las aplicaciones

```
docker compose logs --tail=20 web1 web2
```

Resultado esperado Ambos procesos muestran que escuchan en el puerto 3000.

Paso 3 Inspeccione headers y cuerpo

```
curl -i http://localhost:8080/
```

Resultado esperado Los headers incluyen X-Request-ID y X-Upstream-Addr; el JSON incluye node y requestId.

Relacione el requestId del JSON y del header con el log de NGINX. Registre upstream, upstream_status, request_time y upstream_time. Dibuje los saltos HTTP y explique qué nombres resuelve el DNS de Compose.

Experimento C Balanceo observable

Temas de Unidad 1 Balanceo de carga | Round robin | Salud y observabilidad

Paso 1 Genere una muestra mayor

```
for i in $(seq 1 20); do
  curl -s http://localhost:8080/
  echo
done
```

Resultado esperado Más de una instancia recibe solicitudes.

La evidencia requerida es que más de una instancia reciba trabajo. No se exige alternancia perfecta porque los fallos, reintentos y conexiones pueden alterar la secuencia.

Semana 1 Latencia y fallo parcial

Experimento D Latencia no es fallo

Temas de Unidad 1 Red asíncrona | Latencia | Timeout no prueba fallo | Interacción síncrona

Paso 1 Active la demora solo en web1

```
docker compose -f docker-compose.yml -f compose.delay.yml \
  up -d --force-recreate web1
```

Resultado esperado web1 se recrea con DELAY_MS igual a 2000.

Paso 2 Compare cuatro tiempos

```
for i in $(seq 1 4); do
  time curl -s http://localhost:8080/
  echo
done
```

Resultado esperado Las respuestas de web1 tardan cerca de dos segundos; web2 responde rápidamente.

Paso 3 Quite la demora

```
docker compose up -d --force-recreate web1
```

Resultado esperado web1 vuelve a ejecutarse sin DELAY_MS.

El overlay añade 2000 ms a web1. El endpoint /health queda sin demora para que el health check describa la vida del proceso y no la latencia artificial. Compare los tiempos de web1 y web2 y explique por qué una respuesta lenta no demuestra un fallo.

Experimento E Fallo parcial

Temas de Unidad 1 Fallos parciales | Crash stop | Alta disponibilidad | Failover

Paso 1 Detenga una instancia

```
docker compose stop web1
```

Resultado esperado web1 queda detenido mientras los demás servicios continúan.

Paso 2 Envíe solicitudes durante el fallo

```
for i in $(seq 1 10); do
  curl -sS --max-time 4 http://localhost:8080/
  echo
done
```

Resultado esperado El servicio continúa respondiendo mediante web2. Una primera solicitud puede tardar por el timeout de conexión.

Paso 3 Restaure la instancia

```
docker compose start web1
```

Resultado esperado web1 vuelve a estar disponible después de iniciar.

NGINX descubre el fallo al intentar conectarse. `max_fails` y `fail_timeout` implementan detección pasiva. Un health check de Compose permite observar el contenedor, pero no dirige el tráfico de NGINX.

Entregable de la semana 1

Elemento	Contenido
Evidencia	Identidad, balanceo, tiempos y caída de web1
Diagrama	Cliente a NGINX a web1 o web2
Análisis	Cinco a ocho líneas sobre distribución y fallo parcial
Pregunta puente	Qué ocurre si ambos nodos representan un mismo contador

Semana 2 Estado local y compartido

Esta semana añade estado mutable. El contador local pertenece a cada proceso. El contador compartido reside en state y puede ser consultado por ambos nodos web.

Experimento A Estado local y divergencia

Temas de Unidad 1 Aislamiento de procesos | Coordinación | Consistencia

Paso 1 Detenga y remueva el laboratorio

```
docker compose down
```

Resultado esperado Compose elimina los contenedores y la red del proyecto.

Paso 2 Inicie un estado limpio

```
docker compose up --build -d
```

Resultado esperado Los servicios arrancan con contadores en cero.

Paso 3 Incremente contadores locales

```
for i in $(seq 1 8); do
  curl -s -X POST http://localhost:8080/counter/local
  echo
done
```

Resultado esperado web1 y web2 incrementan variables distintas.

Paso 4 Lea los contadores locales

```
for i in $(seq 1 6); do
  curl -s http://localhost:8080/counter/local
  echo
done
```

Resultado esperado El valor observado depende del campo node.

Ordene las respuestas por node. Si ambos procesos representan un contador lógico único, los valores diferentes muestran ausencia de coordinación.

Experimento B Estado compartido

Temas de Unidad 1 Coordinación y serialización | Estado externalizado | Stateless

Alcance Se observa un único estado compartido. No se implementan replicación ni consenso.

Paso 1 Incremente el contador compartido

```
for i in $(seq 1 8); do
  curl -s -X POST http://localhost:8080/counter/shared
  echo
done
```

Resultado esperado El contador aumenta de forma continua aunque cambie el nodo web.

Paso 2 Lea el contador desde varios nodos

```
for i in $(seq 1 6); do
  curl -s http://localhost:8080/counter/shared
  echo
done
```

Resultado esperado web1 y web2 informan el mismo valor de counter.

Los nodos web delegan la operación a un mismo proceso state. Deben observar el mismo valor. Esto no constituye replicación: solo existe una copia en memoria.

Semana 2 Disponibilidad y arquitectura

Experimento C Disponibilidad web

Temas de Unidad 1 Alta disponibilidad | Redundancia | Failover | Balanceo

Paso 1 Detenga web1

```
docker compose stop web1
```

Resultado esperado La redundancia de la capa web se reduce a una instancia.

Paso 2 Consulte el estado compartido

```
for i in $(seq 1 8); do
  curl -sS --max-time 4 http://localhost:8080/counter/shared
  echo
done
```

Resultado esperado web2 mantiene disponible la ruta hacia state.

Paso 3 Restaure web1

```
docker compose start web1
```

Resultado esperado La capa web vuelve a tener dos instancias.

Experimento D Punto único de fallo

Temas de Unidad 1 Disponibilidad end to end | Crash recovery | Durabilidad | Punto único de fallo

Conexión conceptual El reinicio permite discutir qué estado permanece durable y por qué memoria no equivale a persistencia.

Paso 1 Detenga el servicio de estado

```
docker compose stop state
```

Resultado esperado web1 y web2 siguen activos, pero pierden su dependencia de estado.

Paso 2 Pruebe una lectura

```
curl -i --max-time 4 http://localhost:8080/counter/shared
```

Resultado esperado La API responde HTTP 503 shared_state_unavailable.

Paso 3 Pruebe una escritura

```
curl -i --max-time 4 -X POST http://localhost:8080/counter/shared
```

Resultado esperado La API responde HTTP 503; NGINX no convierte la respuesta en 502.

Paso 4 Restaure state

```
docker compose start state
```

Resultado esperado La ruta vuelve a responder y el contador comienza nuevamente en cero.

Ambas operaciones deben devolver 503 mientras state está detenido. NGINX no reintenta ese 503 en otro nodo web porque ambos dependen del mismo servicio. Al iniciar state, el contador vuelve a cero porque no existe persistencia.

Experimento E Concurrencia sobre el estado compartido

Temas de Unidad 1 Concurrencia | Serialización | Orden de ejecución | Consistencia

Este experimento envía varias escrituras simultáneas. El objetivo es distinguir el orden en que state modifica el contador del orden en que las respuestas llegan a la terminal.

Paso 1 Reinicie state y espere hasta que vuelva a responder

```
docker compose restart state
until curl -fsS http://localhost:8080/counter/shared >/dev/null; do
  sleep 0.2
done
```

Ejecute estas líneas como un solo bloque. La primera reinicia state y devuelve el contador a cero porque el valor solo existe en memoria. El ciclo until consulta la ruta compartida cada 0,2 segundos y termina cuando vuelve a responder; >/dev/null oculta el cuerpo porque aquí solo interesa comprobar disponibilidad.

Resultado esperado state vuelve a estar disponible y el contador comienza en cero.

Paso 2 Envíe veinte incrementos con hasta cinco solicitudes simultáneas

```
seq 1 20 | xargs -n1 -P5 sh -c \
  'curl -sS -X POST -w "\n" http://localhost:8080/counter/shared'
```

Ejecute el bloque completo. seq produce veinte elementos y xargs inicia una solicitud por cada uno. La opción -P5 permite un máximo de cinco solicitudes concurrentes; los números no se envían a la API. Cada POST incrementa el contador una vez y -w agrega un salto de línea a la salida.

Resultado esperado web1 y web2 pueden participar. Los valores deben cubrir del 1 al 20 sin incrementos perdidos, aunque las líneas pueden aparecer en un orden distinto.

Paso 3 Consulte el valor final

```
curl -s http://localhost:8080/counter/shared
echo
```

Resultado esperado La respuesta final informa counter igual a 20.

Preguntas de análisis

- ¿Por qué el orden mostrado en la terminal puede no coincidir con el orden de los contadores?
- ¿Qué componente serializa realmente los incrementos?
- ¿El resultado seguiría siendo correcto si existieran dos réplicas independientes de state?
- ¿Este experimento demuestra persistencia o solamente coordinación dentro de un proceso?

Experimento F Timeout y resultado incierto de una escritura**Temas de Unidad 1 Timeout | Semántica de entrega | Reintentos | Idempotencia**

Este experimento hace que el cliente abandone un POST antes de recibir la respuesta. Después se comprueba si la escritura ocurrió y qué sucede cuando el cliente repite la operación sin saber su resultado.

Paso 1 Prepare un contador limpio y deje solo web1 con demora

```
docker compose restart state
until curl -fsS http://localhost:8080/counter/shared >/dev/null; do
  sleep 0.2
done
docker compose -f docker-compose.yml -f compose.delay.yml \
  up -d --force-recreate web1
docker compose stop web2
docker compose ps
```

Ejecute estas líneas como un solo bloque. state se reinicia para comenzar en cero y el ciclo until espera a que vuelva a responder. El overlay configura una demora de 2000 ms en web1 y --force-recreate aplica la variable DELAY_MS. web2 se detiene para que NGINX solo pueda elegir el nodo lento. Antes de continuar, confirme en ps que web1, state y nginx estén activos y web2 esté detenido.

Resultado esperado El contador está en cero y toda solicitud disponible pasa por web1, que tarda cerca de dos segundos.

Paso 2 Envíe una escritura cuyo cliente solo espere un segundo

```
curl -i --max-time 1 -X POST \
  http://localhost:8080/counter/shared
```

Ejecute este comando por separado. --max-time 1 limita a un segundo la espera de curl. Como web1 demora aproximadamente dos segundos, el cliente termina antes de recibir una respuesta.

Resultado esperado curl informa el error 28 Operation timed out y no muestra una respuesta HTTP completa.

Paso 3 Espere y compruebe si la escritura ocurrió

```
sleep 3
curl -s http://localhost:8080/counter/shared
echo
```

Ejecute estas líneas juntas. sleep permite que web1 continúe después de que curl dejó de esperar. El GET posterior también pasa por el nodo lento, pero conserva el límite normal de NGINX.

Resultado esperado El contador vale 1. La escritura terminó aunque el cliente anterior recibió un timeout.

Paso 4 Repita a ciegas la escritura y consulte nuevamente

```
curl -i --max-time 1 -X POST \
  http://localhost:8080/counter/shared
sleep 3
curl -s http://localhost:8080/counter/shared
echo
```

Ejecute el bloque completo. El segundo POST representa un cliente que interpreta el timeout como fallo y repite la operación. Tras esperar, la consulta permite observar el efecto acumulado.

Resultado esperado El segundo POST también vence para el cliente, pero el contador termina en 2; el reintento duplicó el efecto.

Paso 5 Restaure ambos nodos sin demora

```
docker compose start web2
docker compose up -d --force-recreate web1
```

Ejecute estas líneas juntas. La primera inicia web2 y la segunda recrea web1 usando solo la configuración principal, por lo que elimina DELAY_MS.

Resultado esperado web1 y web2 vuelven a estar disponibles sin latencia artificial.

Preguntas de análisis

- ¿Falló la escritura o solamente falló la entrega de su respuesta?
- Después del timeout, ¿qué información tiene realmente el cliente?
- ¿Por qué repetir automáticamente un POST puede ser peligroso?
- ¿Cómo ayudarían una clave de idempotencia y la deduplicación?
- ¿Sería diferente el problema con una operación naturalmente idempotente?

Discusión sobre consistencia**Temas de Unidad 1 Replicación | Consistencia fuerte y eventual | CAP | Semántica de entrega**

Alcance Estos temas se razonan a partir de lo observado. El laboratorio no simula una partición ni varias réplicas de state.

Pregunta	Concepto que debe aparecer
¿Por qué divergen los contadores locales?	Estado aislado y ausencia de coordinación

Pregunta	Concepto que debe aparecer
¿Qué cambió al introducir state?	Punto común de serialización
¿state está replicado?	No compartido no significa replicado
¿Qué aportarían réplicas?	Redundancia con costo de coordinación
¿Qué implica un timeout en un POST?	Resultado incierto e idempotencia
¿Qué ocurre durante una partición?	Tradeoff entre consistencia y disponibilidad

Actividad de arquitectura

Temas de Unidad 1 SOA y microservicios | Límites de servicio | Costos de distribución

Analice si state debe ser un servicio independiente en un sistema real. La respuesta puede ser afirmativa o negativa, pero debe relacionar los límites con datos, despliegue y fallos.

Criterio	Pregunta para justificar la decisión
Escalado	¿Debe escalar de manera diferente a la API web?
Despliegue	¿Debe evolucionar o publicarse de forma independiente?
Datos	¿Tiene ownership y reglas de consistencia claras?
Fallos	¿Qué latencia y qué nuevo fallo remoto introduce?
Operación	¿Puede observarse y recuperarse sin acoplamiento excesivo?

Entregable de la semana 2

Elemento	Contenido
Evidencia	Estado local, compartido, caída web1, caída state, ráfaga concurrente y escritura que continúa después del timeout
Conceptos	Local, compartido, persistente y replicado
Alta disponibilidad	SPOF y mitigación conceptual
Arquitectura	Diagrama final y límites de servicio
Conclusión	Máximo 150 palabras sobre el problema añadido por el estado

Checklist y solución de problemas

Validación	Criterio
Docker	<code>docker run --rm hello-world</code>
Compose	<code>docker compose version</code> o <code>docker-compose --version</code>
Configuración	<code>docker compose config</code> sin errores
Build	<code>docker compose build</code>
Pruebas	3 pruebas aprobadas
Smoke test	Ambos nodos y contador verificados
Limpieza	<code>docker compose down</code>

Problemas frecuentes

Síntoma	Acción
Permission denied	Aplicar la política del laboratorio o solicitar habilitación
Puerto 8080 ocupado	Revisar <code>ss -ltnp grep ':8080'</code> o cambiar el puerto
502 al inicio	Esperar unos segundos y revisar ps y logs
503 en shared	Revisar state y sus logs
nginx.conf no cambia	Ejecutar <code>docker compose restart nginx</code>
Contenedor recreado no responde	Esperar la actualización DNS y revisar logs de nginx

Limpieza final

```
docker compose down
```

No use `docker compose down -v` como rutina en proyectos reales sin verificar qué volúmenes contienen datos. Este laboratorio no define volúmenes persistentes.